# Git collaboration workflow

i am working on a project, the code is hosted on github, i am working on main, with the entire project being pushed on there, my peer is working on the front end, he is working and he recently pushed his work on a branch, i now want us both to be working on the same version, hence why i told him to push to a branch, i am thinking of these next steps:
- push my latest changes to main
- merge main and his branch
- push the merged to main
- make him pull the entire project again
- we both work on our features
- we merge again and so on

questions:
- does this make sense? is it the correct approach to collaborate on a coding project?
- if so, how do i achieve this step by step

Yes — that workflow is fundamentally correct, and it's very close to how small software teams collaborate using GitHub.

What you are describing is essentially:

- **`main`** = stable/shared version
- each person works on a separate branch
- changes get merged into **`main`**
- everyone syncs from **`main`**

That is the standard collaborative Git workflow.

---

## Recommended Workflow

For a 2-person graduation project, a good workflow is:

## Main ideas

- **`main`** should always contain the latest working version.
- Nobody should directly code on **`main`** long-term.
- Each feature/fix should ideally be done in its own branch.
- Merge completed work into **`main`**.

---

## Your Current Situation

You:

- have local changes on **`main`**

Your peer:

- already pushed frontend work to another branch

Goal:

- combine both versions safely
- continue collaborating without overwriting each other

---

## Correct Next Steps

## STEP 1 — Commit and Push Your Current Work

First make sure YOUR latest code is saved on GitHub.

On your machine:

```bash
git status
```

If there are changed files:

```bash
git add .
git commit -m "Latest backend updates"
```

Then push:

```bash
git push origin main
```

Now GitHub has your newest backend code.

---

## STEP 2 — Get Your Peer's Branch

Fetch all remote branches:

```bash
git fetch
```

See branches:

```bash
git branch -a
```

You should see something like:

```Bash
remotes/origin/frontend-branch
```

## STEP 3 — Merge His Branch Into Main

Stay on main:

```Bash
git checkout main
```

Make sure main is updated:

```Bash
git pull origin main
```

Now merge his branch:

```Bash
git merge origin/frontend-branch
```

## STEP 4 — Resolve Merge Conflicts (If Any)

If both of you edited the same files/lines, Git may say:

```Bash
CONFLICT
```

You then:

- open the conflicted files
- choose which code to keep
- combine both if needed

You'll see markers like:

```
<<<<<<< HEAD
your code
=======
his code
>>>>>>> frontend-branch
```

Remove those markers after fixing.

Then:

```Bash
git add .
git commit
```

Git creates the merge commit.

---

## STEP 5 — Push the Merged Version

```Bash
git push origin main
```

Now:

- `main` contains BOTH your work
- GitHub has the unified project

---

## STEP 6 — Your Peer Syncs

Your peer should now:

```Bash
git checkout main
git pull origin main
```

Now both of you have the SAME version.

---

## IMPORTANT: Better Workflow Going Forward

Instead of both coding directly on `main`, do this:

## YOU

Create a branch:

```Bash
git checkout -b backend-feature-x
```

Work and push:

```Bash
git push origin backend-feature-x
```

## YOUR PEER

Works on:

```Bash
frontend-feature-y
```

## When Finished

Merge branches into `main`.

This is safer because:

- fewer conflicts
- easier debugging
- cleaner history
- prevents accidental overwrites

---

# Recommended Team Workflow (Simple)

## Every day

Before coding:

```Bash
git checkout main
git pull origin main
```

Then create/update your branch:

```Bash
git checkout your-branch
git merge main
```

This keeps your branch updated.

---

## Typical Collaboration Cycle

## Developer A

```Bash
git checkout -b feature-a
```

codes...

```Bash
git add .
git commit -m "Implemented feature A"
git push origin feature-a
```

## Developer B

does same on another branch.

---

## Merge

One person:

```Bash
git checkout main
git pull
git merge feature-a
git merge feature-b
git push
```

## Very Important Rule

Avoid this:

```Bash
git push --force
```

on `main`.

It can overwrite teammates' work.

---

## Best Practice for Your Graduation Project

Since you are using:

- backend/frontend separation
- MVC architecture
- service layers
- database
- external APIs

I strongly recommend:

## Branch Naming

```
feature/frontend-auth
feature/backend-chatbot
bugfix/profile-edit
```

## Optional But Highly Recommended

Use Pull Requests on GitHub instead of merging locally.

Advantages:

- review changes before merge
- see diffs visually
- easier conflict resolution
- safer collaboration

Workflow:

1. Push branch
2. Open Pull Request
3. Review
4. Merge into main

Even for 2 people, this is worth learning.

---

## Minimal Simple Workflow (Good for Students)

## Each developer

```Bash
git checkout main
git pull
git checkout own-branch
```

Work → commit → push branch.

---

## Integrator (one of you)

```Bash
git checkout main
git pull
git merge branch-name
git push
```

## One More Important Recommendation

Protect **`main`** mentally as:

- stable
- deployable
- working

Never leave broken code on **`main`**.

That mindset scales very well into professional software engineering.